

ÉCOLE NORMALE SUPÉRIEURE DE LYON
CENTRE DE RECHERCHE EN INFORMATIQUE, SIGNAL ET
AUTOMATIQUE DE LILLE
(UMR 9189 CNRS, UNIVERSITÉ DE LILLE)

ALGOMUS
COMPUTER SCIENCE
M1 INTERNSHIP

A Model for the Rule-Based Generation of
Choral Music

Student
Romain CARL

Supervisors
Mathieu GIRAUD (DR CRNS)
Ken DÉGUERNE (CR CRNS)

August 25, 2024

Contents

1	Introduction	1
1.1	Project Genesis	1
1.2	Roadmap of Report	2
2	Related Works	2
2.1	Representing Music: Pitches and Chords	2
2.2	Computational Music Analysis	3
2.2.1	Analyzing Musical Form	3
2.2.2	Pattern Mining and Matching	3
2.3	Algorithmic Music Generation	4
2.3.1	Machine Learning and Evolutionary Algorithms	4
2.3.2	Constraints and Rules: Exploring the Solution Space	4
2.3.3	David Cope’s <i>Recombinant Music</i>	5
3	Model Proposal	6
3.1	Notational Conventions	6
3.2	Refinement Trees	6
3.3	Viewpoints and Rules	7
3.4	Single-Pass Model	8
3.5	Refinement Model	11
3.6	Additional Extensions	13
4	Implementation and Example Scenarios	16
4.1	Model	16
4.2	Scenarios	16
4.2.1	Holly Herndon Project	16
4.2.2	Sacred Harp Harmonization	17
5	Discussion and Conclusion	17
	References	18
A	Appendix	21

Musical Glossary

A compilation of short definitions for musical terms used in this report, which will hopefully be of some help to readers with little background in music theory.

ambitus The range of pitches featured in a melody or part . 12, 13, 22

cadence A style-specific sequence of chords that concludes a musical phrase . 17, 24, 25

chorale A hymn tune associated with sacred music; usually refers to four-part harmonizations of these tunes . 4, 5, 6, 15

chord A set of (usually simultaneously-sounding) pitches . 1, 2, 3, 5, 7, 12, 16

clausula Brief pattern of notes used at the end of a phrase; the simultaneous use of different clausulae in different voices makes up a cadence. 8, 12, 14

consonant Consonant pitches or intervals are those that are perceived as well-sounding. In occidental music tradition, typical consonant intervals are the unison, the third, the fifth, the sixth and the octave. . 3

counterpoint The technique of writing seemingly independent melodies that create harmonious music when sounding together . 4

dynamics The loudness of a musical note, as prescribed by the composer in the score. . 2

four-part Adjective that describes music written for four different voices (in choir music, typically: soprano, alto, tenor, and bass) . 1, 16

fugue A complex piece of music where a main theme (*subject*) is repeated and developed by different voices . 3, 4

harmonization The process of adding to a part that contains a fixed melody some other parts so as to obtain a well-sounding ensemble . 2, 4, 5, 15, 17

interval The distance between two pitches; the interval between two identical pitches is the *unison*, that between adjacent pitches the *second*; then come the *third*, the *fourth*, the *fifth*, etc. . 2

motif A short musical pattern . 3

part A single line in a musical composition, written for a specific instrument or voice . 16

phrase A formal unit in music, which can be likened to a sentence in language . 1, 3, 17

pitch A note's height, e.g., A4 or B $\flat$ 2 . 2

score The written notes of all parts of a piece of music . 1

1 Introduction

Writing music is a difficult task. Anyone who has ever attempted it has experienced the struggle of inventing a melody that sounds well both on its own and in the context of other concurrent melodies. Musicians have been harnessing this complex multidimensionality by using sets of intricate *rules* for centuries ([Tym10, Chapter 6]), each set of rules describing a musical *style*.

Given the complexity of the task, it is not surprising that algorithms have been long employed for *symbolic*¹ music generation [Cop91, Chapter 1], and Machine Learning (ML) approaches have been on the rise for the last years [SSK15, HPN17]. For all their strengths, ML methods however still fail to capture some details of certain musical styles [Sze24] that rule-based methods – i.e. approaches relying on the explicit representation of domain knowledge in form of rules – have successfully been imitating for more than 30 years [Ebc90]. Therefore, further investigation of how rule-based methods can be used in music generation, on their own or in combination with ML approaches, seems promising.

1.1 Project Genesis

In January 2024, American artist and composer Holly Herndon² commissioned the Algomus team to generate some musical scores, which were to be used in the process of creating an upcoming exhibition at Serpentine Galleries, London.³ Herndon asked for the pieces to be set for four-part choir and in a style loosely based on that of *Sacred Harp*, a sacred music tradition originating in 18th-century New England [Kel09]. Having contacted *Algomus* in search of a research internship at the same time, I was offered to take part in the project; but since Herndon needed the generations to be completed before the start of my internship, I was only marginally involved in the development of the first model.

This first model⁴ revolves around sequences of *modular generator blocks*. Such a block generates a number of variants for a certain musical element – a line of lyrics, a melodic phrase or a sequence of chords – and outputs the best of these variants according to its *scorers*. Blocks are scheduled one after the other – e.g., to first generate some lyrics, then some rhythms, etc. – where later blocks take as input the output of earlier ones. Generation functions, scorers and block scheduling make up a *scenario*, which describes a musical style and can be seen as a realization of the abstract, style-independent model. In close collaboration and exchange with Herndon, a scenario was crafted by hand in order for the output to meet the artist’s demands. Statistical insights obtained from simple analysis of some *Sacred Harp* collections⁵ and from the musicological community [Kel16] were used in the generation functions.

One can describe this system as a means of creating music “via the interaction of a human initiative [i.e., the creation of rules by hand] and a computational initiative [the application of these rules by the model]” [YLA14], a trait termed *co-creative* in recent literature [KTXT14]. Although this first model proved to be a well-suited tool for this artistic collaboration, as a generic model for producing music in different styles, it can be improved upon in several ways:

First, all constraints it imposes are *flexible*: by means of the scorers, some musical events are incentivized while others are penalized, but none are strictly forbidden. Throughout history, the practice of musical writing has however often been guided by *strict* rules.⁶

¹I.e., on the level of written *notes*, as opposed to approaches that model music as an acoustic signal.

²See https://en.wikipedia.org/wiki/Holly_Herndon

³See <https://www.serpentinegalleries.org/about/press/2024-exhibition-programme-highlights>

⁴Which is accessible under <https://gitlab.com/algomus.fr/sacred-harp>.

⁵As found on <https://www.shapenote.net/>.

⁶E.g., in Renaissance choir music, two voices must not (under any circumstances) be led in parallel fifths [Jep92].

Second, the generation order is fixed: once a block has computed its output, it will never be reconsidered. Yet, the ability to question and revise earlier choices is paramount to musical writing [Ebc90].

Third, its flat representation of temporal structure does not do justice to the hierarchical, branching nature of music as established by [LJ96] and reduces the possible co-creative interactions, since, e.g., a user can not keep the gist of an overall satisfactory melody and only rework some of its sub-parts.

Fourth, no means of setting certain blocks to an unalterable content is provided. However, such scenarios are common in musical writing: e.g., in the standard exercise of *harmonization*, everything may be chosen freely except for a fixed melody.

Last, the model had not been formalized and its inner workings could only be understood by reading the code.

1.2 Roadmap of Report

My internship was about improving on the aforementioned shortcomings by proposing and implementing a *new, generic model for choral music generation*, partially inspired by some found in literature.

Section 2 provides an overview of related works in Computational Music Analysis and Generation and details two approaches to algorithmic music generation that have had the greatest impact on my own model.

Section 3 proposes a new formalized model for music generation, accompanied by examples. A simpler model, roughly corresponding to the one used for the artistic collaboration, is first presented and then gradually extended with several features.

Section 4 covers the implementation as well as experiments on two scenarios: the artistic collaboration and harmonization of a *Sacred Harp* tune.

Although time constraints did not allow for a formal evaluation of the obtained generations, Section 5 lists some methods that could be used to this end, as well as possible future work on the model and potentially interesting scenarios.

2 Related Works

2.1 Representing Music: Pitches and Chords

One of the central problems in computational music processing is that of *representing* music in a machine-readable way. Let us describe some among the most established methods in *Music Information Retrieval (MIR)* and Computational Music Analysis.

Pitches are often represented in *Musical Instrument Digital Interface (MIDI)* encoding, i.e., by the number of their corresponding key on a piano. This mapping is however oblivious to enharmonic spellings, e.g., both $C\sharp$ and $D\flat$ are represented by the same number. Alternative representations, such as Hewlett’s interval-invariant Base-40 System [Hew86] or the *MusicXML* format, which denotes base pitch (C) and chromatic alteration ($\sharp$), avoid this issue. However, pitch is only one among several descriptors of a musical *note*, where other important ones include its onset time in the piece, its duration or its dynamics. Multidimensional vectors have often been employed ([LWM⁺02, Lar14, Las23] among others) to this end, with every one of these parameters corresponding to one dimension.

Several notes make up a melody, for which one possible representation is its rough shape (in terms of going “up” and “down”), e.g., expressed by simple graphs [Hur96]. Yet, melodies can also be described by how their individual pitches relate to the underlying chord, as in [KM07]. This simultaneous coexistence of representations for the same event, that operate on varying

levels of granularity and abstractions, generalizes to the concept of *viewpoints* [CW03] and will be prominently used in Section 3.

Likewise, there exists a plethora of representations for chords and successions of them. E.g., *General Chord Types* [Cam16] label chords in absolute terms by determining their root – i.e., the lowest note among a maximal subset of pairwise consonant ones – while relative systems like Lewin’s interval function [Lew59] are only concerned with intervallic relationships between two chords. Such relative systems may even define entire topological chord spaces, as done in [BA16] by building upon Euler’s *Tonnetz*, a harmonic network which arranges chords according to their common pitches.

2.2 Computational Music Analysis

2.2.1 Analyzing Musical Form

Form – i.e., the structure of a musical piece and how its sub-parts relate to each other – plays a pivotal role in musical practice. Among the formal theories encountered in Computational Music Analysis literature, one in particular stands out as the most widely used: in their *Generative Theory of Tonal Music (GTTM)* [LJ96], Lerdahl and Jackendoff propose a set of cognitive *preference rules* that govern how listeners intuitively group elements of a musical piece, and a *reduction* procedure to combine the resulting groups into a hierarchical structure yielding, on the top level, the “skeleton” of the piece. Several attempts have been made to formalize and implement this theory (e.g., [HHT16]), a task complicated by the fact that many *GTTM* rules conflict with each other or rely on a fair amount of musical intuition on the reader’s side.

Since *grammars* offer simple descriptions of such tree-like structures, *Probabilistic Context-Free Grammars (PCFGs)*⁷ have been trained on corpora of musical phrases in [AG14] to represent unseen phrases in the most compact way. One of the issues inherent to PCFGs is that motivic resemblance between sibling nodes cannot accurately be captured. As a remedy for this, [GS15] introduces parametric grammars, where non-terminals are annotated with parameters representing musical motifs. [Roh11] proposes a comprehensive context-free grammar for generating chord sequences in Jazz, featuring rules on all hierarchical levels, from the entire piece down to individual chord symbols. [KM07] implements a generative interactive system for Jazz melodies based on grammars, where users can systematically explore different musical-surface realizations of the generated grammar trees.

2.2.2 Pattern Mining and Matching

The grammatical approaches to analysis seen so far have in common that they are designed to output comprehensive, ready-to-use analyses that by themselves explain the structure of a piece in all its details. Yet, computational analysis can be useful by merely outputting a structured inventory of musical material, used in a subsequent human analysis ([Mar16]). Among the approaches implementing this idea, *pattern mining* has been the subject of numerous publications.

One of the most exhaustive algorithms for pattern mining is found in [Lar14]: successions of multi-dimensional note objects make up heterogeneous patterns, that are built by successive generalization in a single pass through a piece and grouped by similarity into motivic classes.⁸ Despite its proneness to combinatorial explosion, the method identifies all subject occurrences in fugues. Another widely used approach is that of point-set compression algorithms [LWM⁺02, Mer16]: representing a musical piece with each of its notes a point in multidimensional space,

⁷Which are common in Natural Language Processing (see, e.g., [KM03]). They are obtained from Context-Free Grammars by assigning each production rule $A \rightarrow \beta$ its own conditional probability $P(A \rightarrow \beta \mid A)$.

⁸For instance, the pattern “C, E”, after encountering “G, B”, will be generalized to the more abstract pattern “upward major third leap”.

these algorithms are concerned with finding *maximal translatable patterns*, i.e., subsets of notes that occur multiple translated times within the piece, and using them to produce a compressed description of the piece. These notions have been formalized with morphological operators in [Las23]. A third group of algorithms for pattern mining is formed by those using variants of dynamic programming. The Mongeau-Sankoff algorithm [MS90] describes an efficient way of computing best alignments⁹ between note sequences and thus find local similarities within a piece. [GGLL15] uses a variant of this algorithm to locate occurrences of subjects in fugues, thereby obtaining a formal analysis with the help of some heuristics and a *Hidden Markov Model (HMM)*.

2.3 Algorithmic Music Generation

As seen in the previous Subsection, music generation is linked to its analysis by grammars, that cover both. This Subsection presents other approaches to music generation and discusses two authors in particular (Ebcioglu and Cope) whose methods inspired this work. For a comprehensive taxonomy of systems existing prior to 2017, the reader is referred to [HCC17].

2.3.1 Machine Learning and Evolutionary Algorithms

ML methods make up a large share of publications on the topic, with first applications of simple statistical methods like Markov chains to melody generation dating back to the 1950s [Pin56]. The use of more advanced models since then has included melody generation with *Long-Short-Term-Memory (LSTM)* networks [SSK15] (which stand out for being able to capture long-term structure, also see [PRK⁺10]) or other types of *Recurrent Neural Networks (RNNs)* [HPN17]. Although these models generate music that broadly resembles the style of the Bach chorales they have been trained on, their outputs are consistently flawed, since they contain infractions of the part-writing rules observed in the training corpus, like parallel fifths. For this reason, [Sze24] applies some rule-based post-processing to the output of a Generative Transformer Network. These kind of *hybrid* approaches, which combine the strengths of ML systems with the well-formedness guarantees of rule-based systems, have been one of the motivations for the modular design of our system.

Evolutionary Algorithms (EAs) make up another family of generation algorithms. They work by iteratively selecting, mutating and recombining some candidate solutions to create new ones, before discarding the less desirable according to some *fitness functions*. For music generation, these fitness functions provide a practical way of implementing preference rules. E.g., musical styles like Renaissance counterpoint, that textbooks such as [Jep92] describe with a set of part-writing rules, can be approximated by adapting these rules to the EA framework [PDBB97]. In other instances, hand-crafted rules might be needed, as in [McI94] for Bach chorales.

2.3.2 Constraints and Rules: Exploring the Solution Space

Since fitness values in EAs are numerical, rules in the above approaches might assign low values to an undesirable solution, but can not *exclude* it. This is different in Constraint Logic Programming [JL87], where *constraints* specify hard requirements for possible solutions. Interpreters of languages with CLP support¹⁰ proceed by systematic exploration of the solution space via a *backtracking algorithm*. Applying this approach to harmonization means constructing individual

⁹That is, the minimal cost of a sequence of *non-overlapping* operations (like insertion, deletion, substitution, ...) transforming sequence *A* into sequence *B*. It is not to be confused with the edit distance, where operations may be overlapping [BGJ19].

¹⁰Among such languages, *Prolog III* [Col90] is one of the most notable.

voices by exhaustively enumerating all possibilities for a single note, choosing one among them and proceeding with the next note, until a point is reached where the constraints are unsolvable and backtracking to a previously made decision is required. A simple application of this to a miniature rule-set with 20 harmony and voice-leading rules can be found in [AT91].

Among CLP-like methods, Kemal Ebcioglu’s work on chorale harmonizations in the style of Bach [Ebc90] stands out as exhaustive and of great musical quality. It is implemented in a logic-programming language specifically designed for this purpose and allowing for the formulation of constraints in first-order logic. The approach builds on a *knowledge base* comprising three different types of rules [Ebc90, 155] that operate on assignments of partial solutions (i.e., some generated elements): *production rules* generate possible assignments, *constraints* immediately reject some of them and *heuristics* evaluate how desirable the remaining are, thus determining the order of their exploration. A compilation technique for intelligent backtracking [Ebc90, 159] allows to efficiently identify past ill choices in case of a constraint failure. The rules of the knowledge base operate on one of the three following *viewpoints* [Ebc90, 157]: *chord-skeleton* – representing rhythm-less chord progressions without ornaments – *fill-in* – representing the fully embellished four voices and, by means of several sub-viewpoints, how they relate to each other – and *Schenkerian analysis* – representing more abstract structural melodic features like contour. These viewpoints can be seen as different perspectives on the musical surface, which itself emerges from their combination.

Music generation is then done by arranging the viewpoints in a round-robin-chain and performing a *generate-and-test-loop* on them [Ebc90, 158]: assignments for the partial solution of a viewpoint are generated with the production rules, pruned with the constraints and the best one according to the heuristics is adopted in the new partial solution. This loop is executed until no more production rule can be applied and the next viewpoint in the chain is called. Backtracking can take place within, but also between viewpoints, which highlights the importance of this mechanism to the process.

Given a set of 350 hand-written rules found from empirical observations, treatises and the author’s personal intuitions about Bach’s music [Ebc90, 163], this method generates chorale harmonizations on the level of a “talented student of music who has studied the Bach chorales” [Ebc90, 163].

2.3.3 David Cope’s *Recombinant Music*

Lastly, let us discuss an approach that does not fit into any of the above families, since it eclectically combines various methods: *Experiments in Musical Intelligence (EMI)* is a data-driven system designed by American composer David Cope to generate music in the style of any musical corpus. Due to the high musical quality of its output, it has been commented and argued about in the artistic and scientific community [Cop01]. The program’s *LISP* source code has only been published partially. Its details and the extent to which the published generations have been edited by Cope therefore remain opaque. A rough understanding of the system’s inner working is conveyed by [Cop01, Chapter 4] on the example of chorales in the style of Bach.

In a first step, a database of *beat objects* is extracted from a collection of pieces in the targeted style. This is done by splitting up all pieces into their individual beats, storing each voice’s current and its following pitch, as well as annotating it with some structural tags resulting from a multilevel formal segmentation reminiscent of a GTTM analysis. These structural tags (one of *S*, *P*, *E*, *A* or *C*, which denote abstract formal functions, such as *Statement*, *Preparation*, etc.) are inferred with the help of interval structures in the original melodic lines the beats are taken from [Cop96, Chapter 2].

In a second step follows the actual generation: initially, a corpus piece is selected as structure template for the new piece. Starting with the template’s first beat, new beats are iteratively

queried from the database to match both the destination pitches of the last generated beat and the current structural tag imposed by the template. Matching destination pitches ensures that all part-writing rules observed in the corpus are inherently observed in the generated chorale, without the need to formalize them. Furthermore, transformational recombination allows for the creation of new beats from existing database entries, e.g., by exchanging or transposing voices.

Cope’s approach, which he has coined *recombinant music* [Cop96, Cop91], builds upon the notion of *exchangeability* – exchanging segments (beats in the discussed example) with musically equivalent ones – which has shaped our own model.

3 Model Proposal

A paradigm frequently encountered in the previous Section might be termed *enumerate-select-backtrack*: among some alternatives for a yet-to-be musical element of the generated piece – database entries extracted from a corpus in Cope’s *EMI* or possible assignments determined by logical production rules in CLP – the musically best-fitting is adopted, where this choice might possibly be reconsidered in the course of generation. Hence, the question arises of whether these different approaches can be generalized into a single generic model for music generation, possibly capable of emulating each of the original techniques.

The proposed model has in parts been motivated by such unification ambitions. It can be characterized as a *tree rewriting system* (such as [SFJFD24] or [GMSZ08]) with the rewriting operations determined by a set of musical *rules*. These rules may be arbitrarily complex (or even randomized) and operate on different *viewpoints* describing the musical surface of the emerging piece.

Let us proceed with formally defining the proposed model. This serves the purpose of presenting the idea behind it in a well-defined, but hopefully palatable form for the reader, without having to deal with the technical details of implementation. Therefore, we will try to strike an appropriate balance between (semi-)formal mathematical notation and informal explanations.

3.1 Notational Conventions

We denote by $\mathbb{N}$ the set of natural numbers including 0, by $\mathbb{R}^+$ the set of all non-negative real numbers, by $\mathbb{B} = \{1, 0\}$ the booleans and by $\Sigma^* = \{a_0 a_1 \dots a_k \mid a_i \in \Sigma \ \forall i \in \{0, \dots, k\} \ \wedge \ k \in \mathbb{N}\}$ the set of finite lists over any set Σ . For $w_1 = a_0 \dots a_k, w_2 \in \Sigma^*$, we write $w_1[i] := a_i$ for access of the i –the element with any $i \in \{0, \dots, k\}$ and $w_1 \uplus w_2$ for the concatenation of w_1 and w_2 . We may also write $w_1 = [a_0, \dots, a_k]$. Given a map $f : \Sigma^* \rightarrow \mathbb{R}$ and $w \in \Sigma^*$, we denote by $\text{sort}_{f\downarrow}(w)$ ($\text{sort}_{f\uparrow}(w)$) the list w' containing exactly the elements of w sorted downwards (upwards) with respect to f . We mean by $g : \dots a \dots \mapsto \dots b \dots$ the map $g : \Sigma^* \rightarrow \Sigma^*$ with

$$g(a_0 \dots a_k) := a'_0 \dots a'_k \text{ with } a'_i := \begin{cases} b & \text{if } a_i = a \\ a_i & \text{otherwise} \end{cases}$$

Furthermore, we denote by $\mathbf{1}_a : A \rightarrow \mathbb{B}$ the indicator function w.r.t. some $a \in A$ and by $\mathcal{I}$ the set of all bounded, non-empty intervals on $\mathbb{N}$ of the form $[\alpha, \omega) = \{n \in \mathbb{N} \mid \alpha \leq n < \omega\}$ for $\alpha, \omega \in \mathbb{N}$ and $\alpha < \omega$.

3.2 Refinement Trees

First, we need a means of modeling the *temporal structure* of music. Inspired by the tree structures proposed in [LJ96] or [Roh11], let us represent a piece by a tree of *intervals*.¹¹

¹¹Although the proposed structures might remind the reader of interval trees [de08], they are distinctly different from them, as interval trees store several intervals per node, where refinement trees only store one, and both

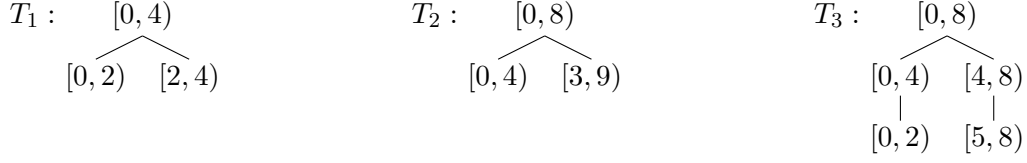


Figure 1: Examples of trees: T_1 and T_3 are refinement trees, whereas T_2 is not. T_1 is also a structure tree.

Definition 1. We say that a list of intervals $S \in \mathcal{I}^*$ **subdivides** an interval $[\alpha, \omega)$ if $S = [\alpha_0, \omega_0) \dots [\alpha_k, \omega_k)$ with

$$\alpha \leq \alpha_0 < \omega_0 \leq \alpha_1 < \omega_1 \leq \dots \leq \alpha_k < \omega_k \leq \omega.$$

If in the above inequality, all $\leq$ are strict equalities, we call this subdivision **gapless**.

Definition 2. We call a **tree** on $\mathcal{I}$ a finite, directed, connected and cycle-free graph $T = (V, E)$ with $V \subset \mathcal{I}$ and $E \subseteq V^2$. Furthermore, we denote for any $I \in V$ by $c(I)$ the list of I 's children sorted by their beginning, and by $root(T)$ the only $I \in V$ for which $\nexists I' \in V, I \in c(I')$.

Definition 3. We call a tree $T = (V, E)$ on $\mathcal{I}$ a **refinement tree** if for all $I \in V$, $c(I)$ subdivides I . If all these subdivisions are gapless, we furthermore call it a **structure tree**.

Since the trees' nodes represent segments of music, one can think of a structure tree as the *complete, multi-level description* of a piece's hierarchical form. Structure trees can be extended into (non-structure) refinement trees by operating on narrower sub-intervals of their leaves, as illustrated by T_3 in Figure 1 – a process we call *refinement*, see Subsection 3.5.

3.3 Viewpoints and Rules

The empty temporal structures we have so far need to be filled with musical content. As mentioned in Section 2, we will use the concept of *viewpoints* [CW03] to represent different aspects of musical events.

Definition 4. A **viewpoint** (or VP) Σ is a set of symbols. We denote by $\mathcal{V}$ the set of all VPs.

Example 1. • $\Sigma_1 = \{G3, A3, B3, C4, D4, E4, F4, G4\}$ is a VP of pitches belonging to the C major scale, spanning one octave from G3 to G4.

- $\Sigma_2 = \{\nearrow, \searrow, \rightarrow\}$ is a VP of melodic contours.
- $\Sigma_3 = \{C, F, a, F\sharp^b7\}$ is a VP of chords.
- $\Sigma_4 = \{\blacktriangle, \blacktriangledown, \circ\}$ is a VP of note durations.

The elements of a VP can be seen as the raw material of which a piece's particular aspect is made. A musical style can be described by a set of VPs and some *rules* that specify how these VPs relate to each other:

Definition 5. For any set $R, \Sigma_1, \dots, \Sigma_l$ VPs (with $l \geq 0$), a **rule** r with image R and domain $(\Sigma_1, \dots, \Sigma_l)$ is a map

$$r : \Sigma_1^* \times \dots \times \Sigma_l^* \rightarrow R$$

Rules can be categorized into three different kinds:¹²

support different operations. Hence the need for our own definitions.

¹²Whose names have been inspired by [Ebc90] except for the *heuristics*, which we call *scorers*.

Definition 6. • For Σ a VP, a **producer** p of Σ is a rule with image $(\Sigma^*)^*$.

- A **constraint** is a rule with image $\mathbb{B}$.
- A **scorer** is a rule with image $\mathbb{R}$.

The purpose of producers is to generate some possible assignments for an interval of a VP, possibly based on the contents of some other VPs; constraints determine whether an assignment for a VP interval (or a combination of assignments for the same interval in different VPs) is valid or not; and scorers assess the desirability of such an assignment by mapping it to a value, which is the higher the more desirable the assignment is. Rules may range anywhere in complexity between large ML models and constants, and even exploit randomness, as the following examples illustrate.

Example 2. • p_1 is a producer of Σ_1 with domain $()$ (i.e., it takes 0 VPs as arguments) that outputs variants for a pitch sequence of length two¹³ by
 $p_1() = \langle \{B3C4, D4C4, D4E4, F4E4\} \rangle$ (where $\langle A \rangle$ denotes a list containing all elements of the set A in random order).

- p_2 is a producer of Σ_1 with domain (Σ_2) that outputs variants for a length-4-melody matching some contour specified by the input and is given by

$$p_2(w) = \begin{cases} [C4D4E4F4, C4E4F4G4] & \text{if } w[0] = \nearrow \\ [G4F4E4D4, G4E4D4C4] & \text{if } w[0] = \searrow \\ [C4E4D4C4, E4F4D4E4] & \text{otherwise} \end{cases}$$

- c_1 is a constraint with domain (Σ_1) and forbids its input to start with B3 by
 $c_1(w) = 1 - \mathbb{1}_{B3}(w[0])$
- c_2 is a constraint with domain (Σ_1, Σ_1) and forbids its two inputs to be the same by
 $c_2(w_1, w_2) = 1 - \mathbb{1}_{w_1}(w_2)$
- s_1 is a scorer with domain (Σ_1, Σ_1) and rewards contrary movement in its two inputs by

$$s_1(w_1, w_2) = \begin{cases} 1 & \text{if } w_1[i] >_p w_1[i+1] \wedge w_2[i] <_p w_2[i+1] \quad \forall i \in \{0, \dots, |w_1| - 2\} \\ 1 & \text{if } w_1[i] <_p w_1[i+1] \wedge w_2[i] >_p w_2[i+1] \quad \forall i \in \{0, \dots, |w_1| - 2\} \\ 0 & \text{otherwise} \end{cases}$$

with $<_p, >_p$ the trivial total order relations on Σ_1 induced by pitch heights.¹⁴

3.4 Single-Pass Model

Let us start by defining a simplified model, roughly corresponding to the one used in the artistic collaboration. Its defining characteristics are that it is *single-pass* (i.e., once an element has been generated, this decision will never be reconsidered), it outputs music with flat structure (i.e., it operates on structure trees of depth 1) and it allows only a single production rule per VP.

Conceptually, we want a model to be a formal description of a *composing procedure*, or *musical style*, capable of generating pieces in that style. In order to motivate the ensuing definitions, let us first *informally* describe an example musical style by stating the properties any of its piece must satisfy:

¹³These sequences are C Major *clausulae*.

¹⁴For now, we take it for granted that $|w_1| = |w_2|$. These kind of assumptions will be made explicit by guards, as introduced in Subsection 3.5.

1. It is *made up* of two simultaneously sounding, rhythm-less voices with pitches from Σ_1 .
2. Both voices *span* 4 pitches, *grouped* by 2.
3. In both of these voices and groups respectively, pitches *are obtained* by choosing among B3C4, D4C4, D4E4 or F4E4.
4. In all groups, the upper voice *must not* start with B3.
5. In all groups, the two voices *must* differ on at least one pitch.
6. In all groups, the two voices *are preferred* to move in opposite directions.

Let us try to capture these properties by defining a model as a list of VPs and a structure tree, with some rules *acting* on its different VPs.

Definition 7. A **model** $\mathcal{M}$ is a tuple (V, T_s, P, C, S) , where:

- $V = \Sigma_0 \dots \Sigma_{k-1} \in \mathcal{V}^*$ is a list of (non-necessarily distinct) VPs,
- T_s is a structure tree of depth 1,
- $P = P_0 \dots P_{k-1}$ a list with each P_i of the form $(i_1, \dots, i_l, p)$ and p a producer of $V[i]$,
- $C = C_0 \dots C_{n_c-1}$ a list with each C_j of the form $(i_1, \dots, i_l, c)$ and c a constraint and
- $S = S_0 \dots S_{n_s-1}$ a list with each S_j of the form $(i_1, \dots, i_l, s, \beta)$ and s a scorer with associated weight $\beta \in \mathbb{R}^+$,

all p, c, s being rules on their respective domains $(V[i_1], \dots, V[i_l])$, with $l \geq 0$.

We denote by $C|_i = \{(i_1, \dots, i_l, c) \in C \mid i = \max(\{i_1, \dots, i_l\})\}$ the set of constraints on Σ_i (and likewise for $S|_i$ and scorers). The use of max ensures that no rule is taken into account until *all* VPs it depends on have been generated.

For example, let the model $\mathcal{M}_1$ be given by

$$\mathcal{M}_1 := (\Sigma_1 \Sigma_1, T_1, (p_1)(p_1), (0, c_1)(0, 1, c_2), (0, 1, s_1, 1))$$

with Σ_1 from Example 1, T_1 from Figure 1 and p_1, c_1, c_2, s_1 from Example 2. Using the following correspondence between the informal properties of the style described above and $\mathcal{M}_1$'s components, it can be *interpreted* as the formal description of that style: 1. (Σ_1, Σ_1) , 2. T_1 , 3. $(p_1)(p_1)$, 4. $(0, c_1)$, 5. $(0, 1, c_2)$, 6. $(0, 1, s_1, 1)$. Let us further bridge this interpretation gap by defining how a model's structure tree is filled with musical content:

Definition 8. Given Σ a VP, a **viewpoint state** S of Σ is a tuple (T, G) , where

- $T = (V, E)$ is a refinement tree and
- $G = (I_0, \ell_0) \dots (I_n, \ell_n)$ is a list of **generations** attached to tree nodes, i.e., (I_i, ℓ_i) with $I_i \in V$ and $\ell_i \in (\Sigma^*)^*$.

We denote by $\mathcal{S}_\Sigma$ the set of all VP states of Σ .

A VP state is a refinement tree with some lists of generations attached to its nodes, where the best-scoring of these generations should come first (see Figures 3 and 5 for examples). Hence, it implicitly also represents a word $w \in \Sigma^*$, which can be constructed by assembling the top generations attached to T 's nodes. This assembling process is made explicit by the following:

Definition 9. For $S = (T, G)$ a viewpoint state and $I \in c(\text{root}(T))$, the **rendered** content $\rho(S, I) \in \Sigma^*$ of S under I is:

$$\rho(S, I) := \ell[0] \quad \text{for } (I, \ell) \in G$$

In the following definitions, we assume the model $\mathcal{M} = (V, T_s, P, C, S)$ with $V = \Sigma_0 \dots \Sigma_{k-1}$ and some associated VP states $S_0 \dots S_{k-1}$ to be implicit parameters, which we do not explicitly notate for the sake of readability.

Notation 1. Let Σ a VP and $I = [\alpha, \omega] \in \mathcal{I}$. Then, for a $w \in \Sigma^*$, we write the **subword** of w in I as:

$$w[I] := a[\alpha] \dots a[\omega - 1]$$

Rules, when applied on an interval I , take as arguments the contents of different VP states under I . We will write this in the following way:

Notation 2. Let r a rule, $I \in \mathcal{I}$ an interval and $i_1, \dots, i_l \in \{0, \dots, k-1\}^*$. Then, we write the **application** of r to the arguments of $S_0 \dots S_{k-1}$ in I as

$$r\{I, i_1, \dots, i_l\} := r(\rho(S_{i_1}, I), \dots, \rho(S_{i_l}, I))$$

or, alternatively, if we want to supply as argument for Σ_i a word $w \in \Sigma_i^*$

$$r\{I, w, i, i_1, \dots, i_l\} := r(f(i_1), \dots, f(i_l)) \quad \text{with } f(i_j) := \begin{cases} w & \text{if } i_j = i \\ \rho(S_{i_j}, I) & \text{otherwise} \end{cases}$$

The second notation is needed because, when applying constraints or scorers to evaluate a new generation w , we want to judge w itself in the context of the other argument VPs, and not the current top generation output by ρ .

Now, let us have the VP states evolve by applying the model's rules on them:

Definition 10. (Operators) Given an interval I , $i \in \{0, \dots, k-1\}$ and letting $\Sigma := V[i]$,

- the **production operator** $\pi_{i,I} : \mathcal{S}_{\Sigma_i} \rightarrow \mathcal{S}_{\Sigma_i} : (T, \dots(I, \ell) \dots) \mapsto (T, \dots(I, \ell') \dots)$ with

$$\ell' := p\{I, i_1, \dots, i_l\} \quad \text{and} \quad (i_1, \dots, i_l, p) = P[i]$$

applies VP i 's producer p to create some generations under I ,

- the **elimination operator** $\varepsilon_{i,I} : \mathcal{S}_{\Sigma_i} \rightarrow \mathcal{S}_{\Sigma_i} : (T, \dots(I, \ell) \dots) \mapsto (T, \dots(I, \ell') \dots)$ with

$$\ell' := [w \in \ell \mid \bigwedge \{c\{I, w, i, i_1, \dots, i_l\} \mid (i_1, \dots, i_l, c) \in C_i\} = 1]$$

applies the constraints in C to eliminate some generations under I and

- the **scoring operator** $\sigma_{i,I} : \mathcal{S}_{\Sigma_i} \rightarrow \mathcal{S}_{\Sigma_i} : (T, \dots(I, \ell) \dots) \mapsto (T, \dots(I, \text{sort}_{f\downarrow}(\ell)) \dots)$ with

$$f : \Sigma^* \rightarrow \mathbb{R} : w \mapsto \sum_{(i_1, \dots, i_l, s, \beta) \in S_i} \beta \cdot s\{I, w, i, i_1, \dots, i_l\}$$

applies the scorers in S to rank some generations under I .

Out of convenience, let us write $\gamma_{i,I} := \pi_{i,I} \varepsilon_{i,I} \sigma_{i,I}$ for the **generation** procedure. For this single-pass model, we define the initial viewpoint state $S_i^0 := (T_s, [(I, []) \mid I \in c(\text{root}(T_s))])$ and a **control stack** – i.e., a list of operations to perform – by simply applying the generation procedure to all intervals:

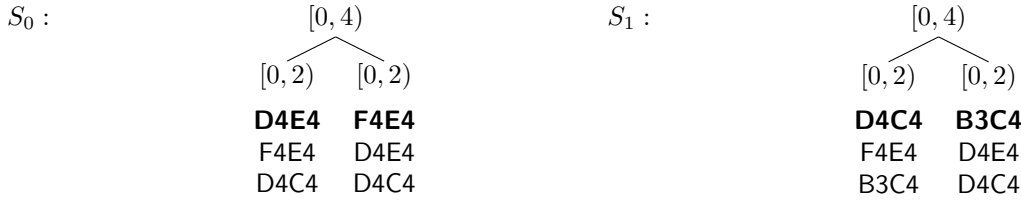
$$\mathcal{C} := \biguplus [\gamma_{i,c_0} \dots \gamma_{i,c_l} \mid i \in 0 \dots k-1] \quad \text{with } c(\text{root}(T_s)) = c_0 \dots c_l$$

Now, let an **execution** on $\mathcal{M}$ simply be obtained by starting with VP states S_i^0 , traversing $\mathcal{C}$, iteratively applying each of its operators $op_{i,I}$ to S_i and returning the list of final VP states. To finally bridge the gap between $\mathcal{M}_1$ and its musical interpretation, Figure 2 shows an example execution and Figure 3 the corresponding final viewpoint states. Observe that in step (6) in Figure 2, constraint c_2 discards D4E4 because it is equal to the content of the upper voice. Likewise, in step (7), the downward motion of D4C4 is preferred over the upward motion of B3C4 because the upper voice moves upwards with D4E4.

S_0	S_1	op.	app. rule	
$(T, \{([0, 2), []], ([2, 4), []])\})$	$(T, \{([0, 2), []], ([2, 4), []])\})$	—	—	(0)
$\dots([0, 2), [\mathbf{D4E4}, \mathbf{B3C4}, \mathbf{F4E4}, \mathbf{D4C4}])\dots$		$\pi_{0,[0,2)}$	p_1	(1)
$\dots([0, 2), [\mathbf{D4E4}, \mathbf{F4E4}, \mathbf{D4C4}])\dots$		$\varepsilon_{0,[0,2)}$	c_1	(2)
$\dots([2, 4), [\mathbf{B3C4}, \mathbf{F4E4}, \mathbf{D4E4}, \mathbf{D4C4}])\dots$		$\pi_{0,[2,4)}$	p_1	(3)
$\dots([2, 4), [\mathbf{F4E4}, \mathbf{D4E4}, \mathbf{D4C4}])\dots$		$\varepsilon_{0,[2,4)}$	c_1	(4)
	$\dots([0, 2), [\mathbf{D4E4}, \mathbf{B3C4}, \mathbf{D4C4}, \mathbf{F4E4}])\dots$	$\pi_{1,[0,2)}$	p_1	(5)
	$\dots([0, 2), [\mathbf{B3C4}, \mathbf{D4C4}, \mathbf{F4E4}])\dots$	$\varepsilon_{1,[0,2)}$	c_2	(6)
	$\dots([0, 2), [\mathbf{D4C4}, \mathbf{F4E4}, \mathbf{B3C4}])\dots$	$\sigma_{1,[0,2)}$	s_1	(7)
	$\dots([2, 4), [\mathbf{F4E4}, \mathbf{D4C4}, \mathbf{B3C4}, \mathbf{D4E4}])\dots$	$\pi_{1,[2,4)}$	p_1	(8)
	$\dots([2, 4), [\mathbf{D4C4}, \mathbf{B3C4}, \mathbf{D4E4}])\dots$	$\varepsilon_{1,[2,4)}$	c_2	(9)
	$\dots([2, 4), [\mathbf{B3C4}, \mathbf{D4E4}, \mathbf{D4C4}])\dots$	$\sigma_{1,[2,4)}$	s_1	(10)

Figure 2: Example execution on model $\mathcal{M}_1$, with final result underneath.

The columns show, from left to right: the two VP states, the currently executed operator and the rules applied in the current step. We have $\mathcal{C} = \gamma_{0,[0,2)} \gamma_{0,[2,4)} \gamma_{1,[0,2)} \gamma_{1,[2,4)}$. σ -steps in which no scorer is applied are omitted. The final result is obtained by setting $\rho(S_0, [0, 2)) \uplus \rho(S_0, [2, 4))$ for the upper and $\rho(S_1, [0, 2)) \uplus \rho(S_1, [2, 4))$ for the lower voice, to a rhythm of quarter notes (the corresponding pitches in the viewpoint states are shown in bold).

Figure 3: Final viewpoint states of the execution of $\mathcal{M}_1$ shown in Figure 2 (after step (10)).

3.5 Refinement Model

We have seen that the single-pass model presented in the previous Subsection generates music without ever reconsidering any choices it makes. This can be problematic in situations where all generations output by a producer fail some constraint. In order to remedy this, let us extend the model by allowing it to *refine*, i.e., regenerate subparts of intervals for generations that fail some constraints. This calls for the following three modifications:

- (a) Unlike the single-pass model, where the control stack did not change during execution, *dynamic control flow* is now needed, since the decision of whether to refine or not depends on the result of preceding production and elimination operations.
- (b) The refinement tree may *grow in depth* during generation, thus possibly evolving into a non-structure tree with gaps.
- (c) In an interactive system, the refinement decision could be made by the user, based on their musical intuition. In the automated system we strive for, a more *fine-grained rule-dispatching mechanism*, capable of identifying constraint-failing subintervals of generations, is needed instead. As a by-product, we will see that this extension allows for endowing a single VP with several producers.

Let the ensuing formal extensions of the single-pass model again be guided by an example musical style. Consider the style described by the piece properties:

1. It is made up of two simultaneously sounding, rhythm-less voices with pitches from Σ_1 .
2. Both voices span 8 pitches, grouped by 4.
3. *The last two pitches* of each voice are clausulae, whereas *the rest* are random pitches.
4. The two voices may not agree *on any pair of consecutive notes*.
5. The two voices are preferred not to cross each other *on any note*.
6. Each of the voices is preferred to have the smallest ambitus possible *on the entire piece*.

For (a), we need to conceive of an operator as a function returning not only an updated viewpoint state, but also a **list of new operations** to be added to the control stack:

```

( $S_0 \dots S_{k-1}, \mathcal{C}$ )  $\leftarrow$  ( $S_0^0 \dots S_{k-1}^0, \mathcal{C}^0$ )
while  $\mathcal{C} \neq []$  do
  | ( $i, op$ )  $\leftarrow pop(\mathcal{C})$ 
  | ( $S_i, new\_ops$ )  $\leftarrow op(S_i)$ 
  |  $\mathcal{C} \leftarrow new\_ops \uplus \mathcal{C}$ 
end
return  $S_0 \dots S_{k-1}$ 

```

For (b), we need to generalize the notion of rendered content of nodes in refinement trees, in order to take into account that descendant nodes might themselves store generations: For $S = (T, G)$, we now write

$$\rho(S, I) := w[[\alpha, \alpha_0]] \uplus \rho(S, [\alpha_0, \omega_0]) \uplus w[[\omega_0, \alpha_1]] \uplus \dots \uplus \rho(S, [\alpha_l, \omega_l]) \uplus w[[\omega_l, \omega]]$$

with $c(I) = [\alpha_0, \omega_0] \dots [\alpha_l, \omega_l]$, $(I, g) \in G$ and $w = g[0]$.

For (c), we introduce **guards** as a means of more fine-grained control over where rules may be applied and where not, in order to capture the restrictions expressed by the words in italics in the above properties. We model guards as maps $g : \mathcal{I} \rightarrow \mathbb{B}$ and adapt Definition 7 accordingly: we add to each of the tuples P_j, C_j and S_j a guard g , which specifies on which intervals the respective rule may be applied. That is, guards offer a way of making assumptions about rule's input's length explicit.

Example 3. • g_1 with $g_1(I) = \mathbb{1}_1(|I|)$ only allows a rule to be applied to intervals of size

1. Similarly, $g_2(I) = \mathbb{1}_2(|I|)$ only permits those of size 2. E.g., a scorer for a single chord must be guarded by g_1 , and one for a pair of pitches by g_2 .
- Let us call $\tilde{g}$ with $\tilde{g}(I) = 1 \forall I \in \mathcal{I}$ the *trivial guard*. Among others, it can be used to guard scorers or constraints that evaluate global properties of VPs, such as a melody's ambitus.
- In the context of a refinement tree T , guards can also take into account positional information. E.g., if $root(T) = [\alpha, \omega]$, let g_3 permit only the *last* 2-element interval on T by $g_3(I) = \mathbb{1}_{[\omega-2, \omega]}(I)$. g_3 captures the restriction expressed in property 2.

Adding guards allows for VPs to be equipped with more than one producer, where each producer targets a specific set of intervals. While this permits to formalize properties such as 2. in the style described above, it also calls for further changes: first, to avoid that some of the structure tree's leaf intervals are discarded by *all* producer guards and end up without any generations, we enforce that P must contain a **default producer** $\tilde{p}$ for every VP, guarded by the trivial guard $\tilde{g}$. Second, we modify the entries of P to be of the form $(i_1, \dots, i_l, i, p, g)$, where i is the index of the VP they operate on.

Let us now modify the operators according to our needs:

- The production operator $\pi_{i,I,p} : (T, \dots(I, \ell) \dots) \mapsto ((T, \dots(I, \ell') \dots), \varepsilon_{i,I})$ with ℓ' as in Definition 10, applies producer p .
- The elimination operator $\varepsilon_{i,I}$ is given in Figure A.1. It partitions I 's generations into the succeeding (g_+) and the failing (g_-), selects the generation with the least constraint failures if none are succeeding and triggers the refining of its faulty subintervals.
- The scoring operator $\sigma_{i,I} : (T, \dots(I, g) \dots) \mapsto ((T, \dots(I, \text{sort}_{f\downarrow}(g)) \dots), [])$ is analogous to the one given in Definition 10, but uses a sorting function f , shown in Figure A.1, that builds on the guard mechanism.
- The **dispatch operator** $\delta_{i,I}$ dispatches VP i 's producers to all subintervals of I to which they apply according to their guards and fills the remaining gaps by assigning the default producer $\tilde{p}$ to them. It is given in Figure A.1.

Let the initial VP states be as in Subsection 3.4 and the initial control stack be given by

$$\mathcal{C}^0 := \biguplus [\delta_{i,c_0} \dots \delta_{i,c_l} \mid i \in 0 \dots k-1] \quad \text{with } c(\text{root}(T_s)) = c_0 \dots c_l$$

For example, let the model $\mathcal{M}_2$ be given by

$$\begin{aligned} \mathcal{M}_2 := & (\Sigma_1 \Sigma_1, T_4, \\ & (0, p_1, g_3)(0, \tilde{p}, \tilde{g})(1, p_1, g_3)(1, \tilde{p}, \tilde{g}), \\ & (0, 1, c_2, g_2), \\ & (0, 1, s_2, 1, g_1)(0, s_3, 1, \tilde{g})(1, s_3, 1, \tilde{g})) \end{aligned}$$

with Σ_1 from Example 1, p_1, c_2 from Example 2, g_1, g_2, g_3 from Example 3, $\tilde{g}$ the trivial guard, T_4 from Figure 5 and furthermore:

- $\tilde{p}$ a producer of Σ_1 with domain $()$ that, if called on I , returns 3 sequences of $|I|$ pitches each, uniformly sampled from Σ_1 ,
- s_2 a scorer with domain (Σ_1, Σ_1) , given by $s_2(w_1, w_2) = \begin{cases} 1 & \text{if } w_1[0] \geq_p w_2[0] \\ 0 & \text{otherwise} \end{cases}$ and
- s_3 a scorer with domain (Σ_1) that rewards little ambitus by $s_3(w) = \frac{1}{\max_i w[i] - \min_i w[i]}$

Again, $\mathcal{M}_2$ can be seen as the formalization of the musical style described above via the following correspondence: 1. $\Sigma_1 \Sigma_1$, 2. T_4 , 3. $(0/1, p_1, g_3)(0/1, \tilde{p}, \tilde{g})$, 4. $(0, 1, c_2, g_2)$, 5. $(0, 1, s_2, 1, g_1)$, 6. $(0/1, s_3, 1, \tilde{g})$. An example execution of $\mathcal{M}_2$ is presented in Figure 4, with the corresponding final VP states in Figure 5. Note that scorer s_2 penalizes voice crossings, but does not strictly forbid them, as the crossing at position 4 demonstrates. In step (6), **C4D4** is preferred over the other alternatives because, considering what has already been generated in $[0, 4)$, it leads to the smallest ambitus **C4-G4**. In step (11), all generations fail constraint c_2 on at least one subinterval; therefore, **G3E4F4D4** (which only fails c_2 in $[1, 3)$) is selected and refinement of the faulty interval is triggered (steps (12)-(15)). In step (21), **F4E4** is discarded because it fails constraint c_2 *both* on $[4, 6)$ and $[3, 5)$.

3.6 Additional Extensions

The refinement model still has a number of musically relevant limitations, which shall be briefly addressed by the following extensions. We will therefore abandon the semi-formal notation used this far because the extensions do not alter the generation procedure's core.

S_0	S_1	op.	app. rule	
$(T, \{([0, 4), []], ([4, 8), []])\})$	$(T, \{([0, 4), []], ([4, 8), []])\})$	-	-	(0)
		$\delta_{0,[0,4)}$	-	(1)
$\dots([0, 4), [D4A3G3G4, C4D4F4G4, G4E4F4F4])\dots$		$\pi_{0,[0,4),\tilde{p}}$	$\tilde{p}$	(2)
$\dots([0, 4), [\mathbf{G4E4F4F4}, C4D4F4G4, D4A3G3G4])\dots$		$\sigma_{0,[0,4)}$	s_3	(3)
		$\delta_{0,[4,8)}$	-	(4)
$\dots([4, 6), [A3D4, C4D4, B3F4])\dots$		$\pi_{0,[4,6),\tilde{p}}$	$\tilde{p}$	(5)
$\dots([4, 6), [\mathbf{C4D4}, B3F4, A3D4])\dots$		$\sigma_{0,[4,6)}$	s_3	(6)
$\dots([6, 8), [F4E4, B3C4, D4C4, D4E4])\dots$		$\pi_{0,[6,8),p_1}$	p_1	(7)
$\dots([6, 8), [\mathbf{F4E4}, B3C4, D4C4, D4E4])\dots$		$\sigma_{0,[6,8)}$	s_3	(8)
		$\delta_{1,[0,4)}$	-	(9)
	$\dots([0, 4), [B3E4F4F4, G3E4F4D4, G4E4A3B3])\dots$	$\pi_{1,[0,4),\tilde{p}}$	$\tilde{p}$	(10)
	$\dots([0, 4), [\mathbf{G3E4F4D4}, G4E4A3B3, B3E4F4F4])\dots$	$\varepsilon_{1,[0,4)}$	c_2	(11)
		$\delta_{1,[1,3)}$	-	(12)
	$\dots([1, 3), [E4F4, G4C4, B3A3])\dots$	$\pi_{1,[1,3),\tilde{p}}$	$\tilde{p}$	(13)
	$\dots([1, 3), [\mathbf{G4C4}, B3A3])\dots$	$\varepsilon_{1,[1,3)}$	c_2	(14)
	$\dots([1, 3), [\mathbf{B3A3}, G4C4])\dots$	$\sigma_{1,[1,3)}$	s_2, s_3	(15)
		$\delta_{1,[4,8)}$	-	(16)
	$\dots([4, 6), [G4E4, E4D4, C4D4])\dots$	$\pi_{1,[4,6),\tilde{p}}$	$\tilde{p}$	(17)
	$\dots([4, 6), [G4E4, E4D4])\dots$	$\varepsilon_{1,[4,6)}$	c_2	(18)
	$\dots([4, 6), [\mathbf{E4D4}, G4E4])\dots$	$\sigma_{1,[4,6)}$	s_2, s_3	(19)
	$\dots([4, 6), [B3C4, F4E4, D4E4, D4C4])\dots$	$\pi_{1,[6,8),p_1}$	p_1	(20)
	$\dots([4, 6), [B3C4, D4E4, D4C4])\dots$	$\varepsilon_{1,[6,8)}$	c_2	(21)
	$\dots([4, 6), [\mathbf{B3C4}, D4E4, D4C4])\dots$	$\sigma_{1,[6,8)}$	s_2, s_3	(22)



Figure 4: Example execution on model $\mathcal{M}_2$, with final result underneath. Columns as described in Figure 2. The initial stack is $\mathcal{C}^0 = [\delta_{0,[0,4)}, \delta_{0,[4,8)}, \delta_{1,[0,4)}, \delta_{1,[4,8)}]$. ε -steps in which no constraint is applied are omitted. The final result is obtained by setting $\rho(S_0, [0, 8))$ for the upper and $\rho(S_1, [0, 8))$ for the lower voice, to a rhythm of quarter notes (the corresponding pitches in the viewpoint states are shown in bold). Pitches and executions steps pertaining to the clausula producer p_1 are shown in red, those pertaining to refinement operations in blue, and all other in black.

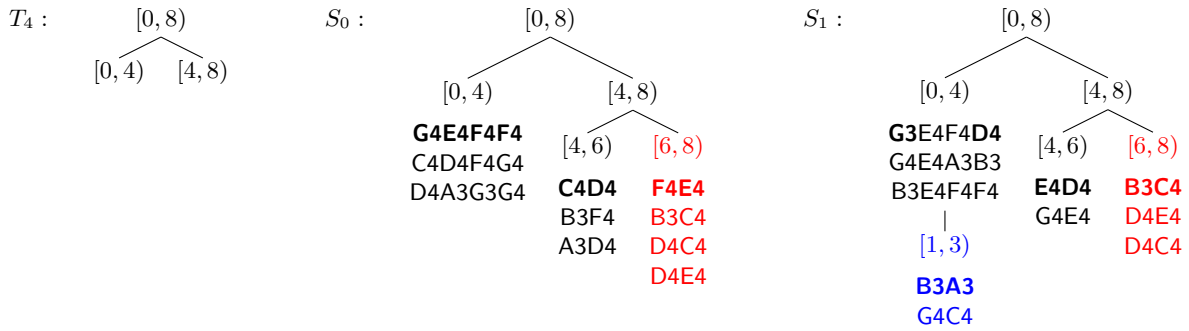


Figure 5: Structure tree T_4 of $\mathcal{M}_2$ and final viewpoint states S_0 and S_1 of its execution shown in Figure 4 (after step (22)). Coloring conventions are the same as in Figure 4.

- **Multi-Level Structure Trees:** So far, we have been operating on “flat” structure trees of depth 1. In order to reflect the deeply nested structure of music established by [LJ96, Roh11], support of arbitrarily deep structure trees is necessary, which can be realized by adapting the dispatch operator δ .
- **Identities Between Tree Nodes:** Resemblance relations between interval nodes on the same level, in the form of *repetition* or *variation* (i.e., an altered repetition), play an indispensable role in musical form. This can be modeled by adding a set of inter-node links $L \subset V \times V$ to a model’s structure tree, that specify which interval nodes copy their generations from each other. Allowing these links to be VP-specific – such that, e.g., two intervals might have common pitches, but possibly different rhythms – results in a mechanism for generating variations.
- **Contextual Information as Rule Input:** When writing producers to match a musical style, one often finds oneself in need of information that goes beyond what can be fetched from other VPs on the current interval, such as the context, before or after the current interval, of the generated VP itself. Enabling rules to optionally take *additional contextual information* as input solves this issue.
- **Flexible-Length Structure:** We assumed the length of a piece to be fixed from the onset of generation. In most scenarios however, the length of generations output by a producer is not pre-determined. This calls for modifying the production operator π in such a way that it can update the refinement tree to reflect the length of the produced generations.
- **Asynchronous Viewpoints:** Ideally, the model should impose as few constraints as possible on the choice of VPs. Complex combinations might include some asynchronous, like (Contour, Rhythm, Pitch), where the VPs Contour and Pitch have different granularity (i.e. one element of Contour maps to several of Pitch, as illustrated by p_2 of Example 2), and the elements of the VP Rhythm intrinsically bear temporal information, whereas the others don’t. A remedy to this lies in indexing the refinement trees not with the previously used ordinals, but with real-valued *temporal positions*, e.g., expressed in quarter beats.
- **Arbitrary Initial Viewpoint States:** In common scenarios, such as the chorale harmonization, some musical elements are initially set and should be unalterable during generation. This can be realized by adding the fixed material to the initial viewpoint states and removing all operations on the concerned intervals from the initial control stack.
- **Backtracking:** The refinement model supports the re-generation of certain intervals, however, it provides no way of backtracking to a decision of a *different viewpoint*, as proposed in [Ebc90]. Yet, this is necessary if, despite refinement, inter-VP constraints still rule out all generations because of an ill choice made in a previously generated VP. This calls for a *backtracking operator* that triggers the re-generation of the other VP’s problematic interval. Subjecting every interval to potential re-generation might endanger some initially set content (see the above extension). By endowing generations with a *fixedness* value, which is maximal for the initial content, and by having the backtracking operator choose the one of the concerned VPs with the minimal generation fixedness, one would avoid this problem, and furthermore introduce a useful quantification of how *certain* the model is about a given decision.
- **Interactivity:** The control stack is currently deterministic. Removing this automatization by allowing any succession of operators on the stack would enable users to interactively steer the course of generation depending on their appreciation of the current VP states.

4 Implementation and Example Scenarios

Having discussed the model in detail, let us broadly present how it was implemented and describe some example scenarios. The code is accessible under <https://github.com/j-w-moebius/algomus-internship>.

4.1 Model

Python 3.11¹⁵ was used for implementation, as this ensured that some code written for and prior to the artistic collaboration could be re-used. If undertaken from scratch, implementations in functional languages with static typing, such as Haskell, might be interesting to explore, since they would probably allow for a result closer to the state-less, functional formalization given in the previous Section.

The core model in its current implementation state (found in the files `ur/src/trees.py` and `ur/src/ur.py`) corresponds to the refinement model as defined in Subsection 3.5, including all additional extensions proposed in Subsection 3.6, except for backtracking and interactivity. Its representation of refinement trees builds on the `anytree` library.¹⁶ Most structures defined in the previous Section have their implementation counterpart in an eponymous Python class. The only notable exception to this is the control stack, which is represented implicitly by the interaction of the respective `generate()` methods of `Model`, `ViewPoint`, `RefinementNode` and `Generator` objects. The latter models the attachment of generated content to tree nodes.

4.2 Scenarios

Two generation scenarios were realized, each consisting of a set of rules, together with a model combining these rules with each other to describe the targeted musical style.

4.2.1 Holly Herndon Project

The first scenario is a re-creation of the one developed for the artistic collaboration with Herndon and therefore indicates that the original model – which roughly corresponded to the single-pass model – can be emulated by the refinement model. Rule set and model are found, respectively, in `ur/src/rulesets/hh.py` and `ur/src/sacred.py`. The model outputs four-part, short choral pieces in a style very loosely based on that of Sacred Harp. Its VPs are:

- `lyr`: a VP of *syllables*, representing the piece’s lyrics common to all parts.
- `rhy`: a VP of *durations*, representing the base rhythm common to all parts.
- `chords`: a VP of *chords*, representing a rough grid of chord progression for the piece.
- `pitchGrid[S|A|T|B]`: 4 VPs of *pitches*, representing the simplified rhythm-less parts.
- `fillIn[S|A|T|B]`: four VPs of *notes* (i.e., durations paired with pitches), representing the fully embellished parts.

Structure trees are chosen at random among the variants listed below, as are the piece’s key, mode and time signature among a number of predefined options. The possible structure trees are best described by strings that label all nodes at depth level 1, from left to right, where $\sim$ denotes a beat-long rest and nodes x and x' are linked on *some* VPs, as described in Subsection 3.6: $A B \sim A' Z$, $A B \sim B' Z$, $A \sim A' Z \sim Z'$ and $A \sim A' B \sim Z \sim Z'$.

The used rules are listed in Table A.1. They feature no constraints, since these were not part of the original model. A sample output is shown in Figure A.2.

¹⁵See <https://www.python.org/downloads/release/python-3110/>.

¹⁶See <https://anytree.readthedocs.io/en/latest/>.

4.2.2 Sacred Harp Harmonization

The second scenario is designed to showcase the advantages of the refinement model over the original model: it corresponds to a *re-harmonization* of *Villulia*.¹⁷ This illustrates how certain VPs can be set to a predefined content and omitted in the generation process. In order to demonstrate refinement in action, it uses constraints, as well as a guarded producer that generates *cadences* at the end of phrases. The rule set, which is again loosely based on the style of *Sacred Harp*, is found in file `ur/src/rulesets/sh.py` and the model in `ur/src/harmonization.py`.

VPs of this scenario are the same as in the previous one, with VPs `rhy`, `lyr`, `pitchGridT` and `fillInT` initially set to match the melody¹⁸ of *Villulia*. The rules by which the other VPs are generated are shown in Table A.2 and a sample output can be found in Figure A.3.

5 Discussion and Conclusion

The previous Section has shown that the proposed model is able to serve as a practical tool in the co-creative generation of choral music, producing output that respects all of its constraints and features the events incentivized by its scorers to a satisfying extent. Its performance and limitations remain to be evaluated, since time constraints did not allow for such an evaluation to be carried out during my internship. Let us therefore mention some methods which could serve this purpose and some further possible future work on the model.

Two approaches can be considered for evaluating the output’s musical quality. The first, *objective evaluation* consists of defining one or several *metrics* – e.g., by counting occurrences of musical events, like certain pitch combinations – and comparing the empirical distribution of this metric on the model’s outputs to its distribution on a musical corpus for the targeted style, by means of tools like the cross-entropy [MMZ23]. A second approach lies in *subjective evaluation*, which can be realized by designing listening experiments, as described in [YL20]. Formal investigation of the model’s limitations could include attempts to emulate, e.g., the expert system of [Ebc90] or parts of Cope’s *EMI* in it. Assessments of the refinement and backtracking procedures’ computational complexities would furthermore be of interest.

Backtracking is yet to be implemented, as mentioned in Section 4. Towards increasing the system’s interactivity, one could aim for a command-line tool enabling the user to edit the control stack during execution. Moreover, a more complete and accurate formalization of the *Sacred Harp* style, taking into account more insights from [Kel09] and [Kel16], would be a reasonable next step in verifying the model’s ability to accurately capture aspects of musical styles.

Members of the *Algomus* team and myself, possibly in collaboration with Holly Herndon, plan to submit two conference papers around the topic: one about the co-creative aspects of the model (possibly for the *International Society for Music Information Retrieval*¹⁹ *ISMIR*) and one about its formalization (possibly for the *International Computer Music Conference*²⁰ (*ICMC*)).

¹⁷Song 56b from the *Sacred Harp* corpus, see <https://sacredharpbremen.org/56b-villulia-3/>.

¹⁸Which always lies in the Tenor in Sacred Harp.

¹⁹See <https://www.ismir.net/>.

²⁰See <https://www.computermusic.org/icmc-conference/>.

References

- [AG14] S. A. Abdallah and N. E. Gold. Comparing models of symbolic music using probabilistic grammars and probabilistic programming. In *Presented at: Joint Sound and Music Computing Conference and International Computer Music Conference 2014, Athens, Greece. (2014) (In press)*., Athens, Greece, September 2014.
- [AT91] M. Aitken and C. P. Tsang. Harmonizing Music as a Discipline in Constraint Logic Programming. *International Computer Music Conference Proceedings*, 1991.
- [BA16] Louis Bigo and M. Andreatta. A Geometric Model for the Analysis of Pop Music. 2016.
- [BGJ19] Henry Boisgibault, Mathieu Giraud, and Florent Jacquemard. What does the Mongeau-Sankoff algorithm compute?, 2019.
- [Cam16] Emiliós Cambouropoulos. The Harmonic Musical Surface and Two Novel Chord Representation Schemes. pages 31–56. January 2016.
- [Col90] Alain Colmerauer. Introduction to prolog III. *Commun. ACM*, 33:69–90, July 1990.
- [Cop91] David Cope. *Computers and musical style*. A-R Editions, Inc., USA, July 1991.
- [Cop96] David Cope. *Experiments in Musical Intelligence*. A-R Editions, 1996.
- [Cop01] David Cope. *Virtual Music: Computer Synthesis of Musical Style*. The MIT Press, May 2001.
- [CW03] Darrell Conklin and Ian Witten. Multiple Viewpoint Systems for Music Prediction. *J. New Music Res*, 24, June 2003.
- [de₀8] More Geometric Data Structures. In Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars, editors, *Computational Geometry: Algorithms and Applications*, pages 219–241. Springer, Berlin, Heidelberg, 2008.
- [Ebc90] Kemal Ebcioglu. An expert system for harmonizing chorales in the style of J.S. Bach. *The Journal of Logic Programming*, 8(1):145–185, January 1990.
- [GGLL15] Mathieu Giraud, Richard Groult, Emmanuel Leguy, and Florence Levé. Computational Fugue Analysis. *Computer Music Journal*, 39:77–96, June 2015.
- [GMSZ08] Blaise Genest, Anca Muscholl, Olivier Serre, and Marc Zeitoun. Tree Pattern Rewriting Systems. In *Automated Technology for Verification and Analysis*, pages 332–346, Séoul, South Korea, 2008.
- [GS15] Mathieu Giraud and Sławek Staworko. Modeling Musical Structure with Parametric Grammars. In Tom Collins, David Meredith, and Anja Volk, editors, *Mathematics and Computation in Music*, pages 85–96, Cham, 2015. Springer International Publishing.
- [HCC17] Dorien Herremans, Ching-Hua Chuan, and Elaine Chew. A Functional Taxonomy of Music Generation Systems. *ACM Computing Surveys*, 50(5):69:1–69:30, September 2017.
- [Hew86] Walter B. Hewlett. CCARH Publications: Base 40 System. <http://www.ccarh.org/publications/reprints/base40/>, 1986.

- [HHT16] Masatoshi Hamanaka, Keiji Hirata, and Satoshi Tojo. Implementing Methods for Analysing Music Based on Lerdahl and Jackendoff’s Generative Theory of Tonal Music. In *Computational Music Analysis*, pages 221–249. Springer International Publishing, Cham, 2016.
- [HPN17] Gaëtan Hadjeres, François Pachet, and Frank Nielsen. DeepBach: a Steerable Model for Bach Chorales Generation, June 2017. arXiv:1612.01010.
- [Hur96] David Huron. The Melodic Arch in Western Folksongs. *Computing in Musicology*, 10, March 1996.
- [Jep92] Knud Jeppesen. *Counterpoint: The Polyphonic Vocal Style of the Sixteenth Century*. Dover Publications, 1992.
- [JL87] J. Jaffar and J.-L. Lassez. Constraint logic programming. In *Proceedings of the 14th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, POPL ’87, pages 111–119, New York, NY, USA, October 1987. Association for Computing Machinery.
- [Kel09] Robert T. Kelley. Sacred Harp Harmony: A Part-Writing Primer for Shape-Note Hymnody. 2009.
- [Kel16] Robert T. Kelley. A Corpus-Based Model of Harmonic Function in Shape-Note Hymnody, 2016.
- [KM03] Dan Klein and Christopher D. Manning. Accurate Unlexicalized Parsing. In *Proceedings of the 41st Annual Meeting of the Association for Computational Linguistics*, pages 423–430, Sapporo, Japan, July 2003. Association for Computational Linguistics.
- [KM07] Robert M. Keller and David R. Morrison. A Grammatical Approach To Automatic Improvisation. In *Proceedings SMC’07*, Lefkada, Greece, July 2007.
- [KTXT14] Anna Kantosalo, Jukka Toivanen, Ping Xiao, and Hannu Toivonen. From Isolation to Involvement: Adapting Machine Creativity Software to Support Human-Computer Co-Creation. June 2014.
- [Lar14] Olivier Lartillot. In-depth motivic analysis based on multiparametric closed pattern and cyclic sequence mining: International Symposium on Music Information Retrieval. *15th International Society for Music Information Retrieval Conference*, 2014.
- [Las23] Paul Lascabettes. *Mathematical Models for the Discovery of Musical Patterns and Structures, and for Performances Analysis*. PhD thesis, Sorbonne Université, November 2023.
- [Lew59] David Lewin. Re: Intervallic Relations between Two Collections of Notes. *Journal of Music Theory*, 3(2):298–301, 1959.
- [LJ96] Fred Lerdahl and Ray S. Jackendoff. A Generative Theory of Tonal Music. In *A Generative Theory of Tonal Music*. The MIT Press, June 1996.
- [LWM⁺02] Kjell Lemström, Geraint Wiggins, David Meredith, The Barn, Middle Drove, and Tydd Giles. Algorithms for Discovering Repeated Patterns in Multidimensional Representations of Polyphonic Music. August 2002.

- [Mar16] Alan Marsden. Music Analysis by Computer: Ontology and Epistemology. In David Meredith, editor, *Computational Music Analysis*, pages 3–28. Springer International Publishing, Cham, 2016.
- [McI94] R.A. McIntyre. Bach in a box: the evolution of four part Baroque harmony using the genetic algorithm. In *Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence*, pages 852–857 vol.2, June 1994.
- [Mer16] David Meredith. Analysing Music with Point-Set Compression Algorithms. In David Meredith, editor, *Computational Music Analysis*, pages 335–366. Springer International Publishing, Cham, 2016.
- [MMZ23] Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: theoretical analysis and applications. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *ICML’23*, pages 23803–23828, Honolulu, Hawaii, USA, July 2023.
- [MS90] Marcel Mongeau and David Sankoff. Comparison of musical sequences. *Computers and the Humanities*, 24:161–175, 1990.
- [PDBB97] John Polito, Jason M. Daida, and Tommaso F. Bersano-Begey. Musica ex Machina: Composing 16th-Century Counterpoint with Genetic Programming and Symbiosis. In *Proceedings of the 6th International Conference on Evolutionary Programming VI*, EP ’97, pages 113–124, Berlin, Heidelberg, April 1997. Springer-Verlag.
- [Pin56] Richard C. Pinkerton. Information theory and melody. *Scientific American*, 194(2):77–86, 1956.
- [PRK⁺10] Marcus T. Pearce, María Herrojo Ruiz, Selina Kapasi, Geraint A. Wiggins, and Joydeep Bhattacharya. Unsupervised statistical learning underpins computational, behavioural, and neural manifestations of musical expectation. *NeuroImage*, 50(1):302–313, March 2010.
- [Roh11] Martin Rohrmeier. Towards a generative syntax of tonal harmony. *Journal of Mathematics and Music*, 5:35–53, March 2011.
- [SFJFD24] Sofía Santiago-Fernández, Joost J. Joosten, and David Fernández-Duque. A tree rewriting system for the Reflection Calculus, July 2024. arXiv:2407.13619.
- [SSK15] Bob L. Sturm, J. F. Santos, and I. Korshunova. Folk music style modelling by recurrent neural networks with long short term memory units. 2015.
- [Sze24] Daniel Szelogowski. *Choral Music Generation: A Deep Hybrid Learning Approach*. PhD thesis, Capitol Technology University, Washington D.C., March 2024.
- [Tym10] Dmitri Tymoczko. *A Geometry of Music: Harmony and Counterpoint in the Extended Common Practice*. Oxford University Press, December 2010.
- [YL20] Li-Chia Yang and Alexander Lerch. On the evaluation of generative models in music. *Neural Comput. Appl.*, 32(9):4773–4784, May 2020.
- [YLA14] Georgios N. Yannakakis, Antonios Liapis, and Constantine Alexopoulos. Mixed-initiative co-creativity. 2014.

A Appendix

```

Function  $\delta_{i,I}((T, G))$ 
   $(V, E) \leftarrow T$ 
   $\mathbf{I} \leftarrow \emptyset$ 
  for  $(i_1, \dots, i_l, i, p, g) \in P$  do
     $S \leftarrow \text{Filt}(\{I' \subseteq I \mid g(I') = 1\})$ 
     $\text{new\_ops} \leftarrow \text{new\_ops} \uplus [\pi_{i,I',p} \mid I' \in S]$ 
     $\mathbf{I} \leftarrow \mathbf{I} \cup S$ 
  end
   $V \leftarrow V \cup \mathbf{I}$ 
   $E \leftarrow E \cup \{(I, I') \mid I' \in \mathbf{I}\}$ 
   $\text{new\_ops} \leftarrow \text{new\_ops} \uplus [\pi_{i,I',\bar{p}} \mid I' \in \text{Gap}(\mathbf{I}, I)]$ 
return  $((V, E), G, \text{new\_ops})$ 

Function  $f(w)$ 
   $\text{score} \leftarrow 0$ 
  for  $(i_1, \dots, i_l, \beta, s, g) \in S$  do
     $\mathbf{I} \leftarrow \text{Filt}(\{I' \in \mathcal{I} \mid I \cap I' \neq \emptyset \wedge g(I) = 1\})$ 
     $\text{subscore} \leftarrow \frac{1}{|\mathbf{I}|} \sum_{I' \in \mathbf{I}} s\{I', w, i, i_1, \dots, i_l\}$ 
     $\text{score} \leftarrow \text{score} + \beta \cdot \text{subscore}$ 
  end
return  $\text{score}$ 

Function  $\varepsilon_{i,I}((T, G))$ 
   $\dots(I, g) \dots \leftarrow G$ 
   $g_+ \leftarrow [w \in g \mid |failing(w)| = 0]$ 
   $g_- \leftarrow [w \in g \mid |failing(w)| > 0]$ 
  if  $g_+ \neq []$  then
    return  $((T, \dots(I, g_+) \dots), \sigma_{i,I})$ 
  end
   $g_- \leftarrow \text{sort}_{|failing| \uparrow}(g_-)$ 
   $\text{new\_ops} \leftarrow [\delta_{i,I'} \mid I' \in failing(g_-[0])]$ 
return  $((T, \dots(I, g_-) \dots), \text{new\_ops})$ 

Function  $failing(w)$ 
   $\text{res} \leftarrow \emptyset$ 
  for  $(i_1, \dots, i_l, c, g) \in C$  do
     $\mathbf{I} \leftarrow \text{Filt}(\{I' \in \mathcal{I} \mid I \cap I' \neq \emptyset \wedge g(I) = 1\})$ 
     $\text{res} \leftarrow \text{res} \cup \{I \in \mathbf{I} \mid c\{I, w, i, i_1, \dots, i_l\} = 0\}$ 
  end
return  $\text{res}$ 

```

Figure A.1: Some operators and related functions of the refinement model: the dispatch operator $\delta_{i,I}$, the elimination operator $\varepsilon_{i,I}$ – which uses the auxiliary function $failing : \Sigma_i^* \rightarrow \mathcal{P}(\mathcal{I})$, which for a word w returns all the intervals in which w fails some constraint (including those that lie partially outside I) – and the auxiliary function f used for the sorting operator, which computes the weighted mean of all scorers on the input generation. Two additional auxiliary functions – Filt and Gap – are used and given in Definitions 11 and 12, respectively.

Definition 11.

$$\text{Filt} : \mathcal{P}(\mathcal{I}) \rightarrow \mathcal{P}(\mathcal{I}) : A \mapsto \{I \in A \mid \nexists I' \in A, I \subseteq I'\}$$

filters out any interval in a set that is covered by some other interval in the set.

Definition 12. For $A = \{I_0, \dots, I_l\} \in \mathcal{P}(\mathcal{I})$ a set of intervals subdividing an interval $I = [\alpha, \omega)$,

$$\text{Gap}(\{[\alpha_0, \omega_0), \dots, [\alpha_l, \omega_l)\}, I) := [\alpha, \alpha_0)[\omega_0, \alpha_1) \dots [\omega_{l-1}, \alpha_l)[\omega_l, \omega)$$

denotes the gaps of A with respect to I .

Name	Arguments	Description	Type
Lyrics	()	Chooses lyrics lines at random from selected Sacred Harp songs, annotated with metrical information.	Producer (1yr)
[Binary Ternary Rhythm	()	Uses a zeroth-order Markov chain to construct rhythms from a set of basic rhythmic elements.	Producer (rhy)
Chords [Major Minor (Extended)]	()	Uses a HMM based on [Kel16] to produce chord sequences.	Producer (chord)
Melody [Major Minor] [S A T B]	()	Uses a first-order Markov chain to construct pitch sequences, inferred from statistical analysis of <i>Sacred Harp</i> corpus.	Producer (pitchGrid[S A T B])
Flourisher ([T B])	(rhy, pitchGrid[S A T B])	Constructs a part from the pitch and duration sequences, randomly applying some embellishment figures to adjacent elements.	Producer (fillIn[S A T B])
ScorerChords	(chord)	Rewards variety and an appropriate number of enriched chords in the sequence.	Scorer (seq.)
ScorerMelodyHarm(Root)	(pitchGrid[S A T B], chords)	Rewards (p, c) if $p \in c$. Non-chord pitches are heavily penalized in first and last elements. So is any bass pitch that is not the chord's root in the last element.	Scorer (1 vs. 1)
ScorerMelody[S A]	(pitchGrid[S A T B])	Rewards appropriate ambitus and contrary motions after leaps and penalizes too many repeated pitches in the sequence.	Scorer (seq.)
ScorerMelodyMelody	(pitchGrid[S A T B], pitchGrid[S A T B])	Penalizes two sequences (m_1, m_2) if they agree on consecutive pitches.	Scorer (2 vs. 2)
ScorerMelodyMelodyCross	(pitchGridA, pitchGrid[S T])	Rewards crossings in two sequences.	Scorer (seq. vs. seq.)
ScorerMelodyMelodyBelow	(pitchGridT, pitchGridB)	Rewards (p_1, p_2) if $p_1 \geq_p p_2$.	Scorer (1 vs. 1)
ScorerRhythmLyrics	(rhy, 1yr)	Rewards (d, s) if adhering to simple association rules like "stressed syllables require longer rhythmic values".	Scorer (1 vs. 1)

Table A.1: Rules used for the Holly Herndon model, as described in Subsection 4.2.1. Column *Type* details the VP that is produced, for producers, or sequence lengths prescribed by the rule's guard, for scorers.

Sacred Harp meets Holly Herndon

S
 Till I Thy name, Thy na ture know. And Je sus by redeem ing blood

A
 Till I Thy name, Thy na ture know. And Je sus by redeem ing blood

T
 Till I Thy name, Thy na ture know. And Je sus by redeem ing blood

B
 Till I Thy name, Thy na ture know. And Je sus by redeem ing blood
 I iii ii IV V I I I iii I vii I vi iii ii I

7
 S
 Till I Thy name, Thy na ture know. a rise and pray, we'll win the day,

A
 Till I Thy name, Thy na ture know. a rise and pray, we'll win the day,

T
 Till I Thy name, Thy na ture know. a rise and pray, we'll win the day,

B
 Till I Thy name, Thy na ture know. a rise and pray, we'll win the day,
 I I ii IV iii ii V I iii ii I I IV V iii I

Figure A.2: A piece generated by the Holly Herndon model, as described in Subsection 4.2.1. Key (D Major), time signature and structure ($A B \sim A' Z$) were randomly chosen.

Name	Arguments	Description	Type
ChordsMinor	()	Same as in Table A.1	Producer (chords)
MelodyMinor [S A B]	()	Same as in Table A.1	Producer (pitchGrid [S A B])
Flourisher ([T B])	(rhy , pitchGrid [S A B])	Same as in Table A.1	Producer (fillIn [S A B])
CadenceChords	()	Generates chord options for cadences (taken from [Kel09]) at the end of phrases.	Producer (chord)
CadencePitches	()	Generates pitch options for cadences (taken from [Kel09]) at the end of phrases.	Producer (pitchGrid [S A B])
MelodyHarm	(pitchGrid [S A B], chords)	Only allows (p, c) if $p \in c$.	Constraint (1 vs. 1)
ScorerChords	(chords)	Rewards variety in chord sequence.	Scorer (Seq.)

Table A.2: Rules used for the Harmonization model, as described in Subsection 4.2.2. Column *Type* details the VP that is produced, for producers, or sequence lengths prescribed by the rule’s guard, for scorers.

Villulia Reharmonized

Figure A.3 displays a musical score for a piece titled "Villulia Reharmonized," generated by the Sacred Harp harmonization model. The score is written for four voices: Soprano (S), Alto (A), Tenor (T), and Bass (B). The lyrics are: "mer cy Oh Thou Son of Da vid, Thus poor blind Bar ti meus prayed, O thers by Thy grace are sa ved, Now to me af ford Thine aid." The Tenor part features the reharmonized melody. Elements generated by the cadence rules are shown in red.

Figure A.3: A piece generated by the Sacred Harp harmonization model, as described in Subsection 4.2.2. The tenor features the reharmonized melody, *Villulia* from the *Sacred Harp* corpus. Elements generated by the cadence rules are shown in red.